

The 7-Day Blind Follow Plan

Custom Software Engineer • Python • React • Agentic AI

YOUR PROFILE

3.8 yrs exp • React + Python + LangGraph

Current CTC: ■6.5 LPA

TARGET

Accenture CSE • Python Role

Expected: ■9.5 – 13 LPA

EXPECTED SALARY RANGE

■9.5 – 13 LPA

Negotiate minimum ■9.5L • Push for ■11–13L • Never reveal current CTC first

INTERVIEW PROCESS

ROUND 1 Coding Test 45 min • 2
Problems Easy–Medium DSA



ROUND 2 Technical Interview
Projects + Python System Design
basics



ROUND 3 HR Interview
Culture fit Salary negotiation

7-Day Overview

Day	Theme	Topics	Time
Day 1	Python Core	OOP, Decorators, Generators, Context Managers	3–4 hrs
Day 2	Python Internals	GIL, Collections, Exceptions, Args/Kwargs, Closures	3–4 hrs
Day 3	DSA Basics	Arrays, Strings, HashMap problems (10–12 questions)	3–4 hrs
Day 4	DSA Timed	Solve 5 problems in 45 min × 2 sessions	3 hrs
Day 5	Project Story	Current project walkthrough + LangGraph narrative	2–3 hrs
Day 6	System Design + HR	REST API, SQL, Agile, Behavioural questions	3 hrs
Day 7	Mock + Review	Full mock interview out loud, weak area revisit	3 hrs

DAY 1

■ Python Core

OOP · Decorators · Generators · Context Managers

OOP — The Big 4

Accenture always asks OOP fundamentals. Know these cold:

Concept	What to say	Quick example
Encapsulation	Bundling data + methods; hiding internals with private vars	<code>self.__balance</code> in <code>BankAccount</code>
Inheritance	Child class reuses parent class properties/methods	<code>class Dog(Animal): ...</code>
Polymorphism	Same method name, different behaviour per class	<code>shape.area()</code> for <code>Circle</code> vs <code>Square</code>
Abstraction	Hide implementation, expose only interface	<code>ABC</code> + <code>@abstractmethod</code>

Decorators — Must Know

```
def my_decorator(func):
def wrapper(*args, **kwargs):
    print('Before function call')
    result = func(*args, **kwargs)
    print('After function call')
    return result
    return wrapper

@my_decorator # same as: greet = my_decorator(greet)
def greet(name):
    print(f"Hello {name}")
```

What interviewer expects: You understand a decorator is a higher-order function that wraps another function.
Real uses: logging, authentication, timing, caching.

Generators — Yield vs Return

```
def count_up(n):
    for i in range(n):
        yield i # pauses here, resumes on next()

gen = count_up(5)
print(next(gen)) # 0
print(next(gen)) # 1

# Why use generators? Memory efficient - doesn't load all at once
```

Context Managers — with statement

```
# Built-in usage
with open('file.txt', 'r') as f:
    data = f.read() # file auto-closed even if exception occurs

# Custom context manager using class
class Timer:
    def __enter__(self): self.start = time.time(); return self
    def __exit__(self, *args): print(time.time() - self.start)
```

Day 1 Checklist

- Explain all 4 OOP concepts out loud
- Write a decorator from scratch
- Write a generator function
- Explain `__enter__` / `__exit__`
- Know difference: `classmethod` vs `staticmethod`
- Know: `__str__` vs `__repr__`
- MRO: Method Resolution Order (C3 linearization)
- `super()` usage in inheritance

DAY 2

Python Internals

GIL · Collections · Exceptions · Args/Kwargs · Closures

Collections Module — High Frequency

Class	Use it when...	Example
Counter	Count frequency of elements	Counter('aabbcc') → {'a':2, 'b':2, 'c':1}
defaultdict	Dict with default value to avoid KeyError	defaultdict(list) for grouping
deque	Fast append/pop from both ends	deque([1,2,3]); d.appendleft(0)
namedtuple	Lightweight immutable data class	Point = namedtuple('Point', ['x', 'y'])
OrderedDict	Dict that remembers insertion order	Rarely needed in Python 3.7+

*args and **kwargs

```
def func(a, b, *args, **kwargs):
# a, b → positional (required)
# *args → extra positional args → tuple
# **kwargs → extra keyword args → dict
print(args) # (3, 4, 5)
print(kwargs) # {'x': 10, 'y': 20}

func(1, 2, 3, 4, 5, x=10, y=20)
```

Exception Handling — Full Pattern

```
try:
result = 10 / 0
except ZeroDivisionError as e:
print(f'Caught: {e}') # specific exception first
except Exception as e:
print(f'Generic: {e}') # catch-all
else:
print('No error occurred') # runs only if NO exception
finally:
print('Always runs') # cleanup code here

# Custom exception
class InsufficientFundsError(Exception): pass
raise InsufficientFundsError('Balance too low')
```

GIL — Global Interpreter Lock

What is it? A mutex in CPython that allows only ONE thread to execute Python bytecode at a time.

Impact: Multi-threading in Python doesn't give true parallelism for CPU-bound tasks. But it's fine for I/O-bound tasks (network calls, file reads).

Solution for CPU-bound: Use `multiprocessing` module — each process has its own GIL.

Closures & Lambda

```
# Closure: inner function remembers outer variable
def multiplier(n):
    def multiply(x): return x * n # 'n' is closed over
    return multiply

double = multiplier(2)
print(double(5)) # 10

# Lambda: anonymous one-liner function
square = lambda x: x ** 2
sorted_list = sorted(data, key=lambda x: x['age'])
```

Day 2 Checklist

- | | |
|---|--------------------------------------|
| ■ Explain GIL in 3 sentences | ■ Create a custom exception class |
| ■ Write Counter / defaultdict usage | ■ Write a closure example |
| ■ Explain *args vs **kwargs from memory | ■ Explain shallow copy vs deep copy |
| ■ Write try/except/else/finally block | ■ Explain mutable vs immutable types |

Accenture's coding test is 2 problems in 45 minutes at Easy–Medium level. These are the exact patterns that appear. Master these, nothing else.

Pattern 1 — String Manipulation

```
# Reverse words in a sentence
def reverse_words(s):
    return ' '.join(s.split()[::-1])

# Check palindrome
def is_palindrome(s):
    s = s.lower().replace(' ', '')
    return s == s[::-1]

# Check anagram
from collections import Counter
def is_anagram(s1, s2):
    return Counter(s1) == Counter(s2)
```

Pattern 2 — HashMap / Frequency Count

```
# Find first non-repeating character
def first_unique(s):
    freq = Counter(s)
    for ch in s:
        if freq[ch] == 1: return ch
    return -1

# Two Sum
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen: return [seen[complement], i]
    seen[n] = i
```

Pattern 3 — Array / Sliding Window

```
# Maximum sum subarray of size k (Sliding Window)
def max_sum_subarray(arr, k):
    window = sum(arr[:k])
    max_sum = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i - k]
        max_sum = max(max_sum, window)
    return max_sum
```

```
# Remove duplicates from sorted array
def remove_duplicates(arr):
    return list(dict.fromkeys(arr))
```

Pattern 4 — Number / Math Logic

```
# FizzBuzz (classic)
def fizzbuzz(n):
    for i in range(1, n+1):
        if i%15==0: print('FizzBuzz')
        elif i%3==0: print('Fizz')
        elif i%5==0: print('Buzz')
        else: print(i)

# Count digits, sum digits
def digit_sum(n): return sum(int(d) for d in str(abs(n)))

# Base conversion
def dec_to_bin(n): return bin(n)[2:] # or format(n, 'b')
```

Day 3 — 12 Problems to Solve

- | | |
|--------------------------------------|-------------------------------------|
| ■ Reverse a string without slicing | ■ Find duplicate in array |
| ■ Check if a string is a palindrome | ■ FizzBuzz |
| ■ Check if two strings are anagrams | ■ Fibonacci (iterative + recursive) |
| ■ Find first non-repeating character | ■ Factorial (iterative + recursive) |
| ■ Two Sum (HashMap approach) | ■ Count vowels in a string |
| ■ Maximum sum subarray of size k | ■ Remove duplicates from list |

DAY 4

■■ DSA Timed Practice

45 min × 2 sessions — Simulate real test

HOW TO PRACTICE TODAY

Set a 45-minute timer. Pick 2 problems below. Write full code from scratch — no autocomplete, no hints. Aim for: Problem 1 fully solved + Problem 2 at least partial.

Do this TWICE. Morning session + Evening session.

Session 1 Problems (Morning)

- Given a string, find longest substring without repeating characters
- Given an array, return all pairs that sum to a target
- Implement a stack using a list with push/pop/peek/is_empty
- Reverse a linked list (using a simple Node class)

Session 2 Problems (Evening)

- Given a list of words, group anagrams together
- Find the majority element in an array (appears > n/2 times)
- Implement binary search on a sorted array
- Merge two sorted arrays without using sort()

Key Python Tricks for Coding Tests

```
# Sort list of dicts
sorted(items, key=lambda x: x['score'], reverse=True)

# Flatten a 2D list
flat = [x for row in matrix for x in row]

# Zip two lists into a dict
d = dict(zip(keys, values))

# Most common element
Counter(arr).most_common(1)[0][0]

# Enumerate (index + value)
for i, val in enumerate(arr): ...

# Check all / any
all(x > 0 for x in nums) # True if all positive
any(x < 0 for x in nums) # True if any negative
```

Accenture Coding Test Tips

- Always write the full program — no pre-existing code is provided on the test platform.
- Pass at least 1 fully + 1 partially — that's the passing bar.
- Handle edge cases: empty list, single element, negative numbers.

- **No negative marking** — always submit even partial code.
- **Think out loud** in the technical interview — say your approach before coding.

DAY 5

■ Project Storytelling

Your current work is your biggest weapon

At Accenture, the technical interview revolves around YOUR projects. The interviewer wants to know what you built, why, and how. This day is about crafting that story perfectly.

The STAR Framework for Project Questions

S — Situation	What was the context/problem?
T — Task	What was your specific responsibility?
A — Action	What did YOU do, step by step?
R — Result	What was the measurable outcome?

Your LangGraph Project — Ready-to-Use Script

S: We had a large legacy application with hundreds of JSP and Angular files sharing global CSS — any change risked breaking multiple apps.

T: My responsibility was to modernize the codebase and eliminate shared CSS dependencies to make each app independently deployable.

A: I built a LangGraph pipeline that automatically analyzed all JSP files, identified which CSS rules were actually used by each app, and separated the shared CSS into app-local files. I used Stylelint for CSS remediation, OWASP Dependency Check for JAR vulnerabilities, and deterministic tooling over LLM-based approaches for accuracy and speed.

R: The pipeline processed files automatically that would have taken weeks manually. CSS isolation improved and the codebase became significantly more maintainable with clear app boundaries.

Likely Project Questions + How to Answer

Q: What is LangGraph and why did you use it?

A: LangGraph is a library built on top of LangChain that lets you model AI workflows as directed graphs with nodes and edges. I used it because our modernization had multiple sequential steps — CSS parsing, linting, dependency checks — and LangGraph let me represent each step as a node with clear state management and conditional branching.

Q: Why did you choose deterministic tools over LLM for this?

A: LLMs introduce variability — two runs might produce different outputs. For code transformations, CVE checks, and CSS parsing, we need 100% reproducibility and integration with real CVE databases. Tools like OWASP Dependency Check give us that. LLMs are great for reasoning and generation, not rule-based transformation.

Q: What was the hardest challenge in this project?

A: The hardest part was identifying which CSS rules were shared vs app-specific across hundreds of files without breaking existing functionality. I had to build a dependency graph to trace CSS class usage per app before safely splitting them.

Day 5 Checklist

■ Practise STAR answer for LangGraph project (out loud)

■ Practise STAR answer for React work

■ Know your biggest technical challenge + how you solved it

■ Prepare 2 examples of code review / team collaboration

■ Explain your tech stack in 60 seconds

■ Know what Agile ceremonies you participated in

Accenture is a service company — system design here is NOT Google-level. They want basics: REST, database design, caching concept, and Agile.

System Design Basics to Know

REST API Design

HTTP verbs: GET (read), POST (create), PUT (update), DELETE. Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauth, 404 Not Found, 500 Server Error. Stateless — server doesn't store client state.

SQL Essentials

Normalization (1NF, 2NF, 3NF) — remove redundancy. Indexes — speed up SELECT, slow down INSERT/UPDATE. Joins: INNER, LEFT, RIGHT, FULL. Transactions: ACID properties.

Caching

Store frequently-read data in memory (Redis) to reduce DB load. Cache invalidation strategies: TTL (time-to-live), write-through, write-behind.

SQL vs NoSQL

SQL: structured, ACID, relations (MySQL, PostgreSQL). NoSQL: flexible schema, horizontal scale, eventual consistency (MongoDB, Redis). Choose SQL for financial data; NoSQL for user activity feeds, logs.

Agile/Scrum

Sprint (2-week cycle), Daily Standup, Sprint Planning, Retrospective. Roles: Product Owner, Scrum Master, Dev Team. Artefacts: Product Backlog, Sprint Backlog, Increment.

HR Questions — Exact Scripts

Q: Tell me about yourself.

A: I'm a software engineer with 3.8 years of experience, specialising in Python backends and React frontends. Currently I'm building LangGraph-based AI pipelines for legacy application modernization. I'm drawn to Accenture because of the scale of problems you work on and the opportunity to build expertise across multiple domains and clients.

Q: Why are you leaving your current company?

A: I've had a strong learning experience, but I'm at a point where I want to work at larger scale, contribute to enterprise-grade systems, and grow into an SME role. Accenture's depth across industries and focus on technology transformation aligns perfectly with where I want to go next.

Q: What is your expected CTC?

A: Based on my experience and market research for this role and level, I'm expecting between ₹11 and ₹13 LPA. I'm open to discussing the full compensation structure.

Q: Where do you see yourself in 3 years?

A: I see myself as a Subject Matter Expert in Python and AI-based solutions, leading technical decisions on projects and possibly mentoring junior engineers. I'd love to have contributed to at least 2–3 successful client transformations by then.

Q: Describe a conflict in your team and how you resolved it.

A: We had a disagreement on whether to use LLM-based code transformation vs deterministic tooling for our pipeline. I collected data on accuracy, reproducibility and speed for both approaches, presented it to the team, and we aligned on deterministic tools. The key was making it a data decision, not a personal opinion.

DAY 7

■ Mock Interview + Review

Full simulation — out loud, timed, no notes

THE RULE FOR TODAY: No notes. No looking things up. Speak every answer out loud as if someone is in front of you. Record yourself if possible.

Morning — Full Technical Mock (90 min)

- Set 45-min timer → Solve 2 medium DSA problems from scratch
- Explain OOP: give real example from your own projects
- Write a decorator for logging function execution time
- Explain generators — when would you use them over a list?
- Design a simple REST API for a task management system (endpoints only)
- Explain the difference between SQL JOIN types with example
- Walk through your LangGraph project using STAR format (timed: 4 min max)
- What happens when you type a URL and press Enter? (basic networking)

Afternoon — HR + Salary Mock (30 min)

- Tell me about yourself (target: 90 seconds, no more)
- Why Accenture? (be specific, not generic)
- What's your expected CTC? (practise saying the number confidently)
- Describe your biggest technical challenge
- Where do you see yourself in 3 years?

Evening — Weak Area Revisit (60 min)

Go back to whichever day's checklist has the most unchecked boxes. Spend the last hour plugging those gaps.

Interview Day Checklist

- | | |
|---|--|
| ■ Keep LangGraph project story under 4 min | ■ Salary: say ■11–13L, never go below ■9.5L |
| ■ Have 2 strengths + 1 weakness ready | ■ Don't reveal current CTC unless forced |
| ■ Know Accenture's 5 core values (look them up) | ■ Join video call 5 min early; test mic + camera |
| ■ Have questions to ask the interviewer | ■ Keep a glass of water nearby |

Python Quick Reference

Pin this page. Review before the interview.

Concept	One-liner / Key point
List comprehension	<code>[x*2 for x in range(10) if x%2==0]</code>
Dict comprehension	<code>{k: v for k, v in items.items() if v > 0}</code>
Lambda	<code>fn = lambda x, y: x + y</code>
Map / Filter	<code>list(map(fn, iterable))</code> <code>list(filter(fn, iterable))</code>
Sorted with key	<code>sorted(lst, key=lambda x: x['age'])</code>
Enumerate	<code>for i, val in enumerate(lst, start=1): ...</code>
Zip	<code>for a, b in zip(list1, list2): ...</code>
Unpack	<code>a, *rest = [1,2,3,4] → a=1, rest=[2,3,4]</code>
String join	<code>', '.join(['a','b','c']) → 'a, b, c'</code>
String split	<code>'a,b,c'.split(',') → ['a','b','c']</code>
F-string	<code>f'Hello {name}, you are {age} years old'</code>
Ternary	<code>x = 'yes' if condition else 'no'</code>
None check	<code>if x is None: ...</code> (not: <code>if x == None</code>)
Type hints	<code>def greet(name: str) -> str: ...</code>
Dataclass	<code>@dataclass class Point: x: float; y: float</code>
copy vs deepcopy	<code>import copy; b = copy.deepcopy(a)</code>

You have React + Python + LangGraph. That's a rare stack for a service company interview. Accenture interviewers are not expecting DSA geniuses — they want confident engineers who can explain their thinking, talk about their work, and write clean code. You are already that person. Now go prove it.